第2章 关系模型

西安交通大学计算机科学与技术学院　　任雪斌

xuebinren@mail.xjtu.edu.cn

目录

2.1 基本概念

2.2 关系代数

2.3 关系演算

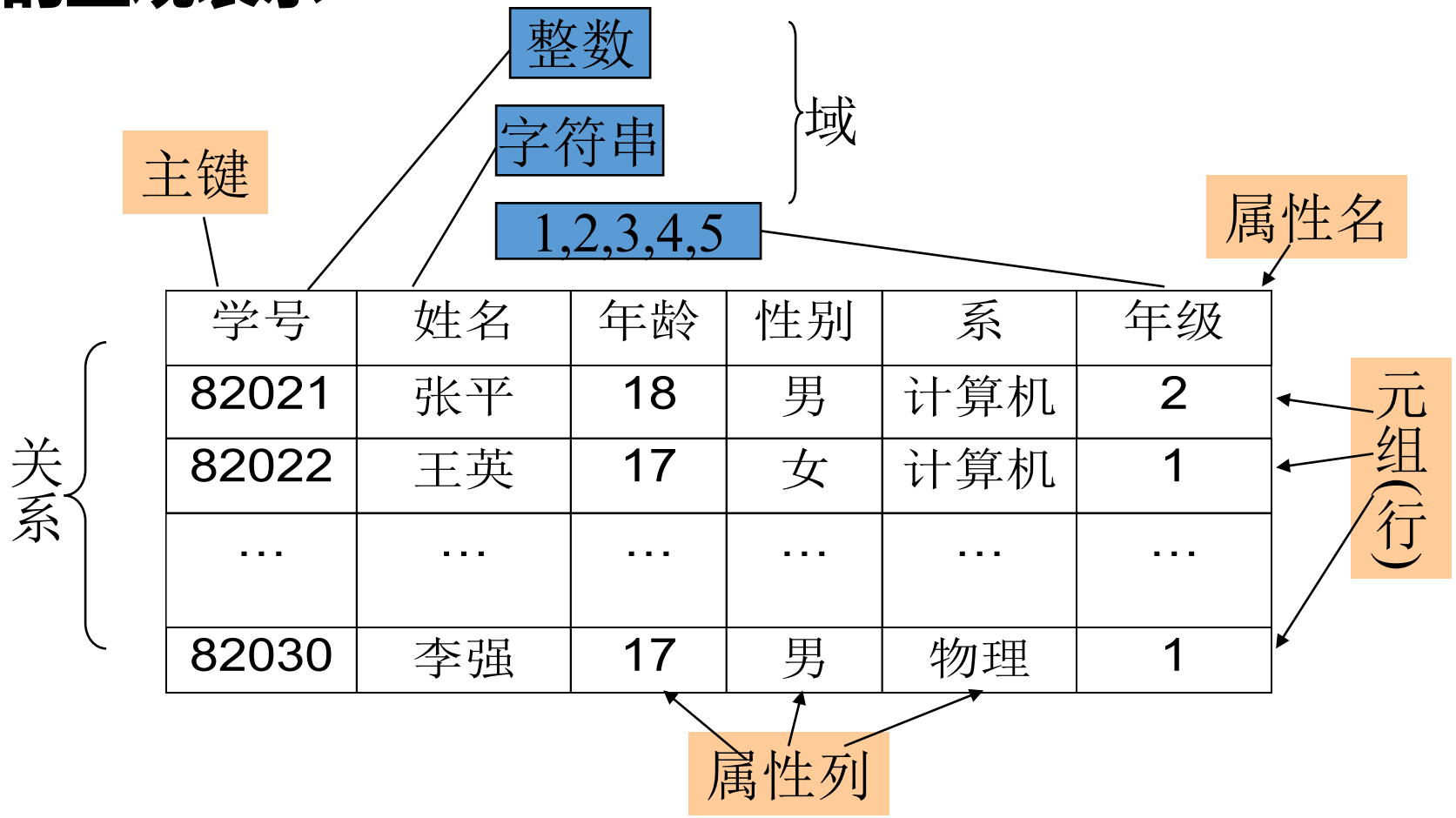
2.1 基本概念

➢关系模型的基本概念

- 关系模型是以集合论中的关系(relation)概念为基础发展起来的数据模型。关系模型理论由IBM公司San Jose研究实验室E.F.Codd率先提出《Communication of the ACM》 “A Relational Model of Data for Large Shared Data Banks”
- 早期代表系统
  - System R：由IBM研制
  - INGRES：由加州Berkeley分校研制
- 目前主流的关系数据库管理系统软件产品包括:
  - Oracle， IBM DB2 UDB， MS SQL Server
  - Sybase， Informix
  - PostgreSQL
  - MySQL
  - Access
  - GBase，Kingbase，OpenBASE，DM7…

2.1 基本概念

➢关系模型的直观表示



关系名：学生登记表

关系模式：学生（学号，姓名，年龄...）

2.1 基本概念

➢关系的定义

- 定义2.1：域(domain)是一组值的集合，同一个域中的所有值均应具有相同的数据类型
- *例2-1： D1={袁玲，吴丹，刘杰} D2={男，女} D3={18,19}*
- 其中：D1(姓名)、D2(性别)和D3(年龄)为域名，分别表示“人员”关系中姓名、性别和年龄的可取值范围
- D1的基数m1=3，D2的基数m2=2，D3的基数m3=2
- 域中元素一般无排列次序，如：D2={男，女}={女，男}

2.1 基本概念

➢关系的定义

- 定义2.2：域$D_1, D_2, \ldots, D_n$上的笛卡儿积(Cartesian product)是一个集合：

$$D_1 \times D_2 \times \cdots \times D_n = \{(d_1, d_2, \cdots, d_n) | d_i \in D_i, 1 \leq i \leq n\}$$

- 其中允许$D_i = D_j$且$i \neq j$，将该集合中的每一个元素$(d_1, d_2, \ldots d_n)$称为一个元组(tuple)，元组中的每一个值d_i称为一个分量(component)，有n个分量的元组称为n元组

2.1 基本概念

➢关系的定义

- *例2-2: 已知三个域： D1={袁玲, 吴 丹 ， 刘 杰 } D2={ 男 ， 女 } D3={18,19}*
- 则D1、D2和D3的笛卡尔积可以表示成一张二维表，如表2-1所示。

D1	D2	D3
袁玲	男	18
袁玲	男	19
袁玲	女	18
袁玲	女	19
吴丹	男	18
吴丹	男	19
吴丹	女	18
吴丹	女	19
刘杰	男	18
刘杰	男	19
刘杰	女	18
刘杰	女	19

2.1 基本概念

➢关系的定义

- 该笛卡尔积的基数为3×2×2=12，即共有12个元组，(袁玲、男、18)和(吴丹、女、19)就是元组，“袁玲”、“女”、“吴丹”、“19”就是分量。
- 有时一个元组也被称为一条记录

2.1 基本概念

➢关系的定义

- 定义2.3：$D_1 \times D_2 \times \cdots \times D_n$上的任意一个子集均是定义在域$D_1, D_2, \cdots D_n$上的一个关系(relation)，记为R
- 由n个域构成的关系通常称为n元关系，关系中的每个元素即是这个关系的元组

姓名	性别	年龄
袁玲	男	18
吴丹	女	19
刘杰	男	18

2.1 基本概念

➢关系的定义

- 由于域可以相同，为了加以区分，必须为每列起一个名字，称为属性(Attribute)
- 定义2.4：为关系的每个列所起的名字称为关系的属性，一般可表示为 A，B，C …
- n元关系必有n个属性，表的任意一列对应一个属性，属性的名称称为“属性名”，具体元组的属性取值称为“属性值”
- *例：学生关系有学号、姓名、性别、系名、年龄等属性*

2.1 基本概念

➢关系的定义

- 定义2.5：如果一个关系中的某个属性或属性集能够唯一的确定一个元组，则称该属性（集）是这个关系上的超键（super key，SK）；如果将超键中的任一属性去掉后剩余的属性集不能唯一标识一个元组，则称该属性集是关系上的候选键（candidate key，CK）；通常从候选键中选择一个使用，这个候选键称为关系的主键（primary key，PK）
- *例 学生（学号,姓名,性别,系名,年龄,……）*

2.1 基本概念

➢关系的定义

- （学号,姓名） 超键

  学号 候选键 主键

- 一般情况下，如不加特别说明，键即指主键。如果关系R1中的某个属性集是另外一个关系R2的候选键，那么该属性集对于关系R1而言是它的外键（foreign key，FK）

- *例 学生表 (学号，姓名，年龄,……)*

  课程表 (课程号，课程名，学分,……)

  选课表 (学号，课程号，成绩,……)

2.1 基本概念

➢关系模式

- 定义2.6：关系模式（relation schema）是对关系的型的描述，可以表示为：

$$R(U, D, DOM, I, F)$$

- 其中，U是R的属性集合$\{A_1, A_2, \dots, A_n\}$，D是属性的取值范围，即域的集合$\{D_1, D_2, \dots, D_n\}$，DOM是U到D的映射集合$\{A_1 \to D_1, A_2 \to D_2, \dots, A_n \to D_n\}$，I是完整性约束规则集，$F$是函数依赖集合
- 习惯上将关系模式简记为$R(A_1 / D_1, A_2 / D_2, \dots, A_n / D_n)$或者$R(A_1, A_2, \dots, A_n)$

2.1 基本概念

➢关系模式

- 关系模式可以表示为：$R(U, D, DOM, I, F)$
- *例子：有一个关系模式为：*$STUDENT(U, D, DOM, I, F)$
  - *属性集合* $U = \{sno, name, age\}$ *//理解为表的字段集合*
  - *域集合* $D = \{char, int\}$ *// 理解为表中字段的类型集合*
  - *属性到域的映射集合* $DOM = \{dom(sno) = dom(name) = char, dom(age) = int\}$ *// 理解为每个字段具体的取值*
  - *完整性约束集合* $I = \{sno! = Null, IsUnique(sno) = True, 18 <= age <= 22\}$ *// 理解为表字段取值的限制*
  - *函数依赖集合* $F = \{sno --> name, sno --> age\}$ *// 理解为表字段的关系*

2.1 基本概念

➢关系模式与关系

- 关系可看作是关系模式在某一时刻的状态或内容，也就是说，关系模式是型，关系是值
- 关系模式是静态的、稳定的，而关系是动态的、随时间不断变化的，因为关系操作在不断地更新着数据库中的数据

2.1 基本概念

➢关系的特征

- 关系中不允许出现相同的元组（元组不重复）
- 关系中各个属性必须有不同的名字，不同的属性可来自同一个域，即他们的分量可以取自同一个域（属性不重复）
- 关系中元组的顺序(即行序)是无关紧要的，在一个关系中可以任意交换两行的次序（元组间无序）
- 关系中属性的顺序是无关紧要的，即属性的顺序可以任意交换（属性间无序）
- 同一属性名下的各个属性值必须来自同一个域，是同一类型的数据。
- 关系中每个分量必须是不可分的数据项，或者说所有属性值都是原子的，是一个确定的值，而不是值的集合，即不可“表中有表” 。属性值可以为空值，表示“未知”或“不可使用”。

2.1 基本概念

➢关系的类型

- 在关系数据库中，关系有以下三种类型：
  - 基本表：实际存在的表，对应实际存储数据的逻辑表示
  - 查询表：对基表查询得到的结果表
  - 视图：从基本表或其他视图中导出的表

2.1 基本概念

➢关系数据库

- 在一个给定的应用领域中，所有关系的集合构成一个关系数据库

2.1 基本概念

➢关系数据库的型与值

- 关系数据库的型: 关系数据库模式，对关系数据库的描述。
- 关系数据库模式包括:
  - 若干域的定义
  - 在这些域上定义的若干关系模式
- 关系数据库的值: 关系模式在某一时刻对应的关系的集合，简称为关系数据库

2.1 基本概念

➢关系的完整性

- 关系模型的完整性（integrity）规则是为了保证数据的正确性和相容性而在关系上施加的约束条件

2.1 基本概念

➢关系的完整性

- 域完整性约束
  - 元组分量在某个属性上的取值应在其值域之内；元组是否能在某个属性上取空值null，由该属性的语义决定
- 实体完整性约束
  - 元组在键上的取值不可重复，且不能为null
  - 按照键的定义，如果不同的元组在键上取相同值或空值，则将无法区分两个不同的元组

2.1 基本概念

➢关系的完整性

- 参照完整性约束
  - 属性集FK是关系R上的外键，属性集CK是关系S上的候选键，FK引用CK，R和S可以是不同的关系，也可以是相同的关系。
  - 参照完整性约束要求R上的元组t在FK属性上的取值t[FK]必须是如下两种情况之一：
    - 等于关系S中某个元组在候选键CK上的值
    - 取空值null

2.1 基本概念

➢关系的完整性

- 用户定义的完整性约束
  - 用户定义的完整性是针对某一具体关系数据库的约束条件，它反映某一具体应用所涉及的数据必须满足的语义要求
  - 关系模型应提供定义和检验这类完整性的机制，以便用统一的系统的方法处理它们，而不要由应用程序承担这一功能

2.1 基本概念

➢关系模型

- 关系模型(Relation Model)由一组相互联系的关系组成，是用二维表结构来表示实体与实体间的联系的模型
- 关系模型由关系数据结构、关系操作集合、关系完整性约束三部分组成，如图所示：

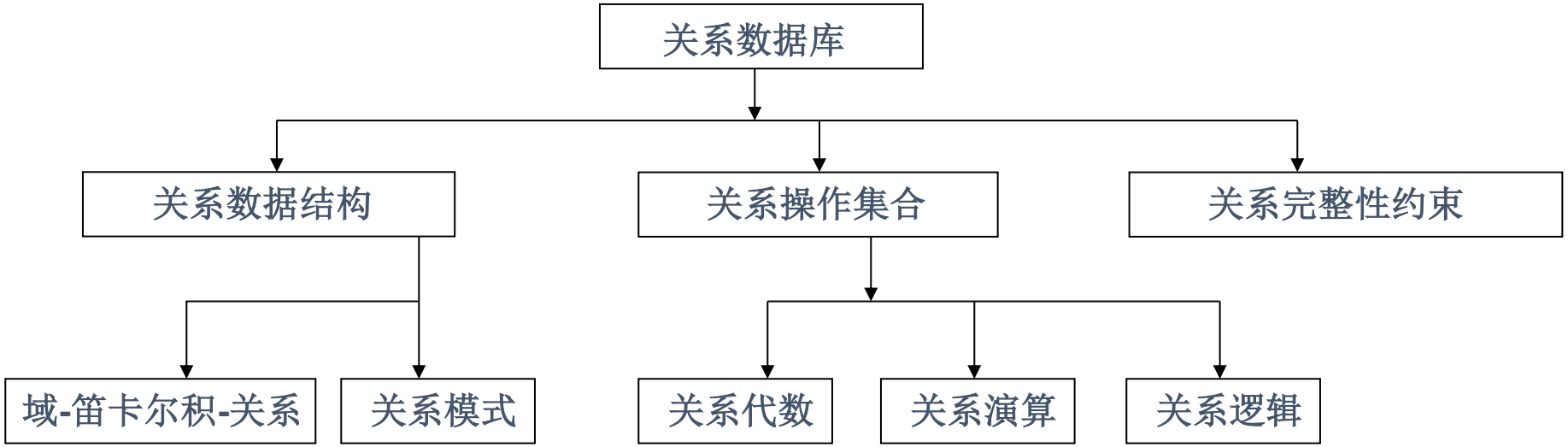


图 关系模型的组成部分

2.1 基本概念

➢关系模型

- 关系操作集合是指关系模型提供的一组完备的高级关系运算，这些关系运算支持对数据库的各种操作。关系运算通常分成关系代数和关系演算
- 为简化分析，有时候也使用简化的关系模型，如图所示

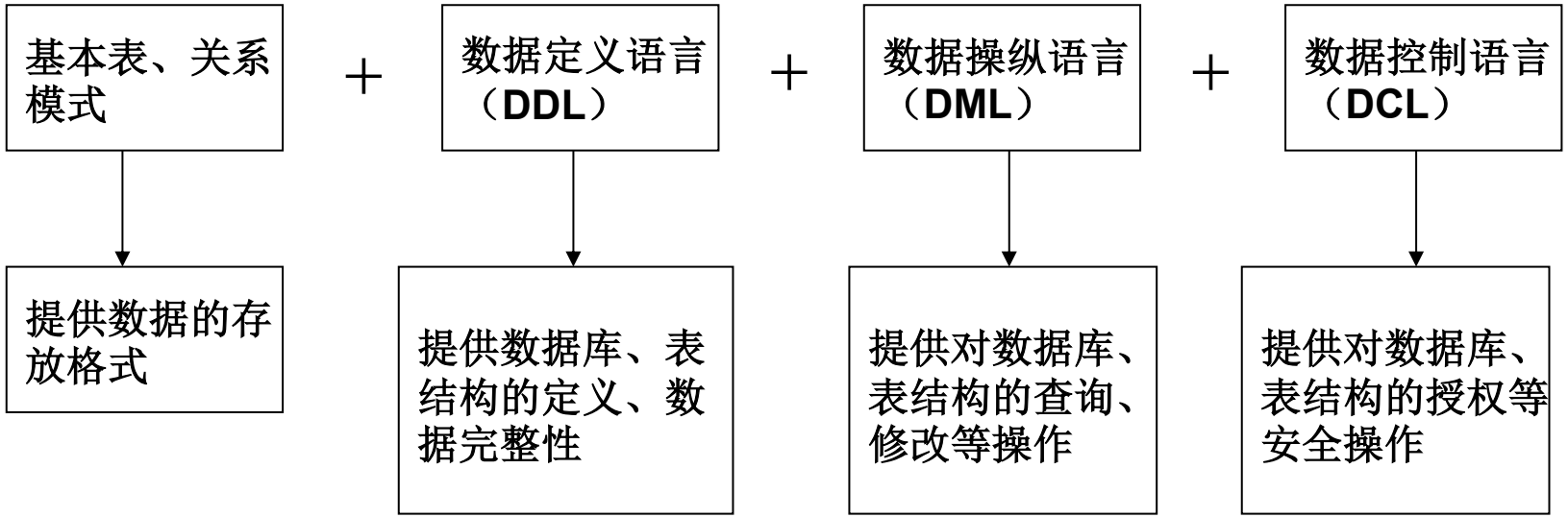


图 简化的关系模型

2.1 基本概念

➢关系模型的优点

- 与其它数据模型相比，关系模型突出的优点如下：
  - 关系模型提供单一的数据结构形式，具有高度的简明性和精确性
  - 关系模型的逻辑结构和相应的操作完全独立于数据存储方式，具有高度的数据独立性
  - 关系模型使数据库的研究建立在比较坚实的数学基础上
  - 关系数据库语言与一阶谓词逻辑的固有内在联系，为以关系数据库为基础的推理系统和知识库系统的研究提供了方便

2.2 关系代数

➢关系代数的介绍

- 关系运算是关系数据库的数学基础，分为关系代数和关系演算两大类
- 关系代数中运算对象是关系，运算结果也是关系，运算符包括四类：
  - 集合运算符：∪(并)，-(差)，∩(交)，×(广义笛卡尔积)
  - 专门的关系运算符：σ(选择)，π(投影)，⋈(连接)，÷(除)
  - 算术比较符：>(大于)，≥(大于等于)，<(小于)，≤(小于等于)，=(等于)，≠(不等于)
  - 逻辑运算符：∧(与),∨(或),┐(非)

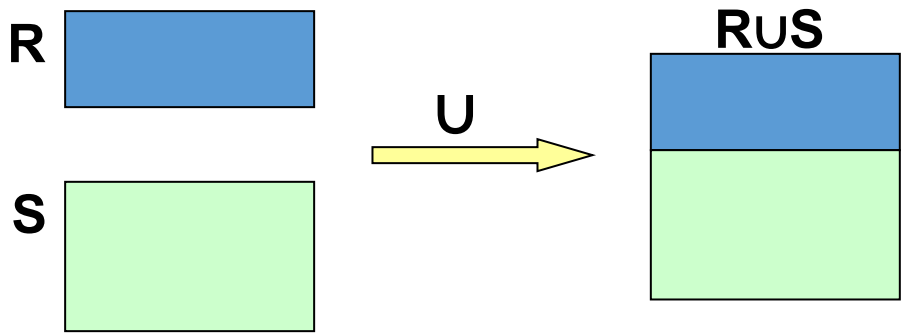
2.2 关系代数

➢传统的集合运算

- 并（union）
  - R、S都是n元关系，且各自的对应属性具有相同的域，则关系R和S并的结果R∪S仍为n元关系，表示为：

$$R \cup S = \{t | t \in R \vee t \in S\}$$

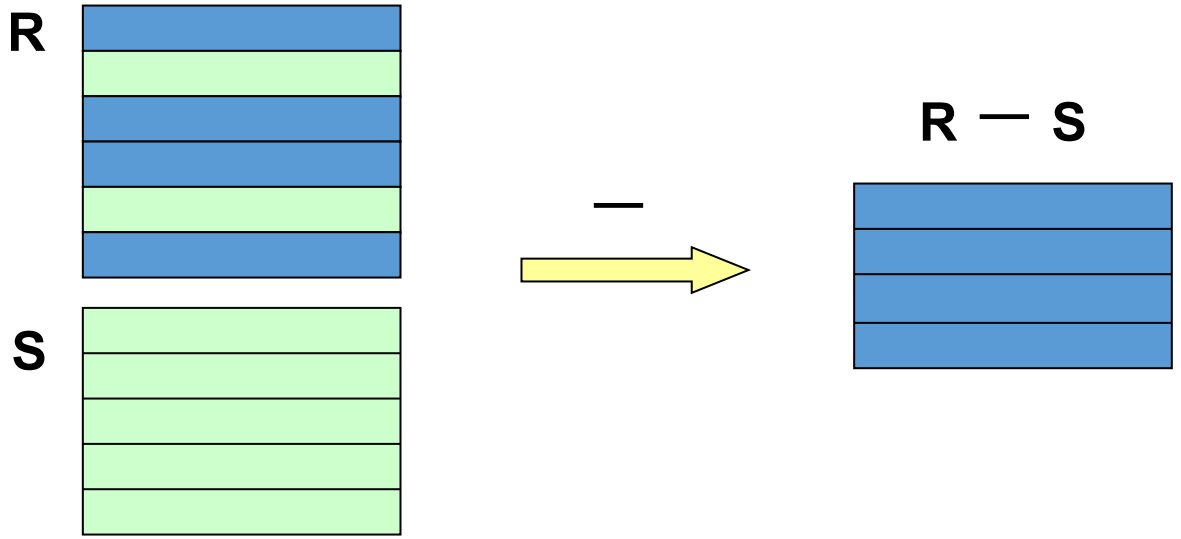
  - 满足并运算要求的两个关系是“并兼容”的



2.2 关系代数

➢传统的集合运算

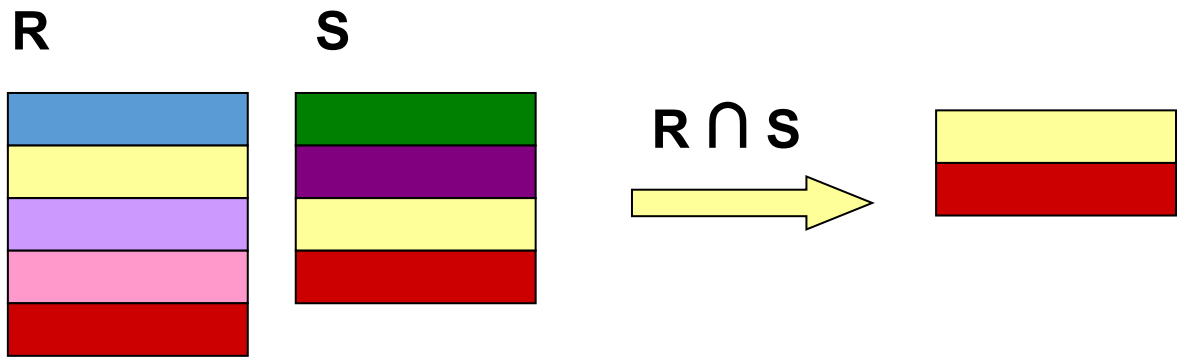
- 差（difference)
  - R和S是并兼容的n元关系，R与S的差R - S仍为n元关系，表示为：$R - S = \{t | t \in R \wedge t \notin S\}$
    即R - S由那些仅出现在R中且不出现在S中的元组构成



2.2 关系代数

➢传统的集合运算

- 交（intersection）
  - R和S是并兼容的n元关系，R与S的交$R \cap S$仍为n元关系，表示为：$R \cap S = \{t \mid t \in R \wedge t \in S\}$ 即$R \cap S$由那些既在R中出现又在S中出现的元组构成



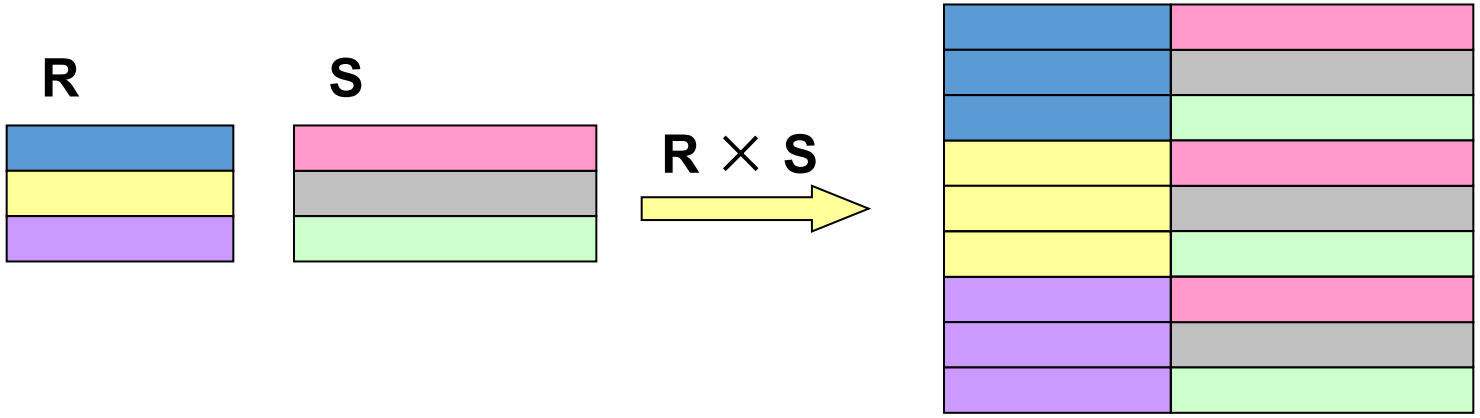
2.2 关系代数

➢传统的集合运算

- 笛卡儿积（Cartesian product）
  - m元关系R与n元关系S的笛卡儿积是（$m+n$）元关系，且每个元组的前m个分量是R的一个元组，后n个分量是S的一个元组，可以表示为：

$$R \times S = \{t(m+n) \mid t(m) \in R \wedge t(n) \in S\}$$

  - 如果R中共有i个元组，S中共有j个元组，则$R \times S$一定有$i \times j$个元组



2.2 关系代数

➢传统的集合运算

A	B	C
a	b	c
d	e	f
g	h	i

(a) $R(A,B,C)$

A	B	C
a	b	c
j	k	l
m	n	p

(b) $S(A,B,C)$

A	B	C
a	b	c
d	e	f
g	h	i
j	k	l
m	n	p

(a)$R \cup S$

A	B	C
d	e	f
g	h	i

(b)$R-S$

A	B	C
a	b	c

(c)$R \cap S$

2.2 关系代数

➢传统的集合运算

A	B	C
a	b	c
d	e	f
g	h	I

(a) $R(A,B,C)$

A	B	C
a	b	c
j	k	l
m	n	p

(b) $S(A,B,C)$

A	B	C	A	B	C
a	b	c	a	b	c
a	b	c	j	k	l
a	b	c	m	n	p
d	e	f	a	b	c
d	e	f	j	k	l
d	e	f	m	n	p
g	h	i	a	b	c
g	h	i	j	k	l
g	h	I	m	n	p

(d)$R \times S$

2.2 关系代数

➢专门的关系运算

- 选择（selection）

$$\sigma F(R) = \{t | \ t \in R \wedge F(t) = true\}$$

  - 其中σ是选择算符，表示进行选择运算，F是选择条件，最简单的选择条件形如$X\theta Y$或者$X\theta c$，X，Y是属性名，c是常量，θ是算术比较运算符，包括＞，≥，＜，≤，＝，≠等。可以由简单的选择条件通过逻辑运算符构成更为复杂的条件，逻辑运算符包括∧，∨和¬。

2.2 关系代数

➢专门的关系运算

- Relation r

A	B	C	D	
α	α	1	7	>5
α	β	5	7	>5
β	β	12	3	≤5
β	β	23	10	>5

$\sigma_{A=B \wedge D>5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10

2.2 关系代数

➢专门的关系运算

A	B	C
a1	b1	c1
a1	b2	c2
a2	b2	c3
a3	b3	c3
a4	b4	c5

(a)关系R

A	B	C
a1	b1	c1
a1	b2	c2
a3	b3	c3

(b) $\sigma_{A='a1' \vee B='b3'}(R)$

A	B	C
a2	b2	c3
a3	b3	c3

(c) $\sigma_{A>'a1' \wedge C='c3'}(R)$

2.2 关系代数

➢专门的关系运算

- 投影（projection）

$$\prod_x (R) = \{t[x] | t \in R\}$$

  - 其中，X是关系R的一个属性子集，t[X]表示关系R上的元组t在属性子集X上的分量
  - 由于关系的属性间是无序的，因此X中的属性顺序可以和它们在R中的出现顺序不一致。此外，需要注意的是因为关系中不允许有重复元组，因此在投影后需要将可能出现的重复元组进行合并

2.2 关系代数

➢专门的关系运算

Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2

2.2 关系代数

➢专门的关系运算

A	B	C
a1	b1	c1
a1	b2	c2
a2	b2	c3
a3	b3	c3
a4	b4	c5

R

A	B
a1	b1
a1	b2
a2	b2
a3	b3
a4	b4

(a) $\Pi_{A,B}(R)$

A
a1
a2
a3
a4

(b) $\Pi_{A}(R)$

B	C
b1	c1
b2	c2
b2	c3
b3	c3
b4	c5

(c) $\Pi_{B,C}(R)$

2.2 关系代数

➢专门的关系运算

- 连接（join）
  - 连接连接是笛卡儿积和选择的复合运算，可以表示为：

$$R \bowtie_{A\theta B} S = \{rs | r \in R \wedge s \in S \wedge r[A]\theta S[B]\} = \sigma_{A\theta B}(R \times S)$$

  - 其中，⋈是连接运算符，$A\theta B$是连接条件，θ是算术比较运算符。$R \bowtie_{A\theta B} S$ 表示将关系R和S在各自的属性A，B上满足θ条件的元组连接起来。θ条件可以有多个，它们之间用逻辑算符连接。这样的连接一般被称为“θ连接”。特别的，如果θ是“等于”比较，称这样的连接运算是“等连接”或“等值连接”

2.2 关系代数

➢专门的关系运算

- 连接（join）
  - 自然连接
    - 消除了重复属性的等值连接，称为“自然连接”。表示如下：

$$R \bowtie S = \pi_{Attr(R) \cup (Attr(S)-A)}(\sigma_{R.A=S.A}(R \times S))$$

    - 其中$Attr(R)$是关系R的属性全集

2.2 关系代数

➢专门的关系运算

- 连接（join）

A	B	C
1	2	3
4	5	6
7	8	9

R

D	E
3	4
7	6

S

A	B	C	D	E
1	2	3	3	4
1	2	3	7	6
4	5	6	7	6

$R \bowtie_{B<D} S$

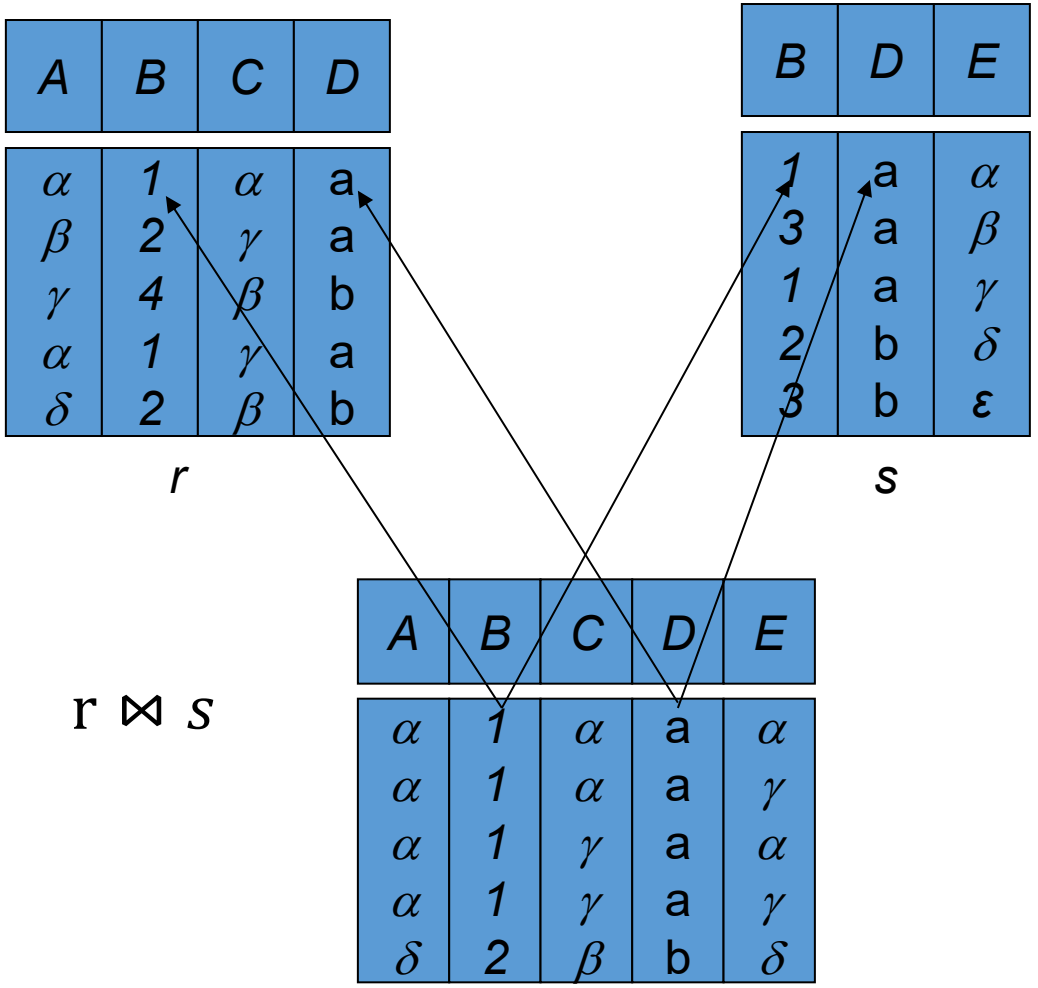
A	B	C	D	E
1	2	3	3	4
1	2	3	7	6

$R \bowtie_{B<D \wedge C<E} S$

2.2 关系代数

➢专门的关系运算

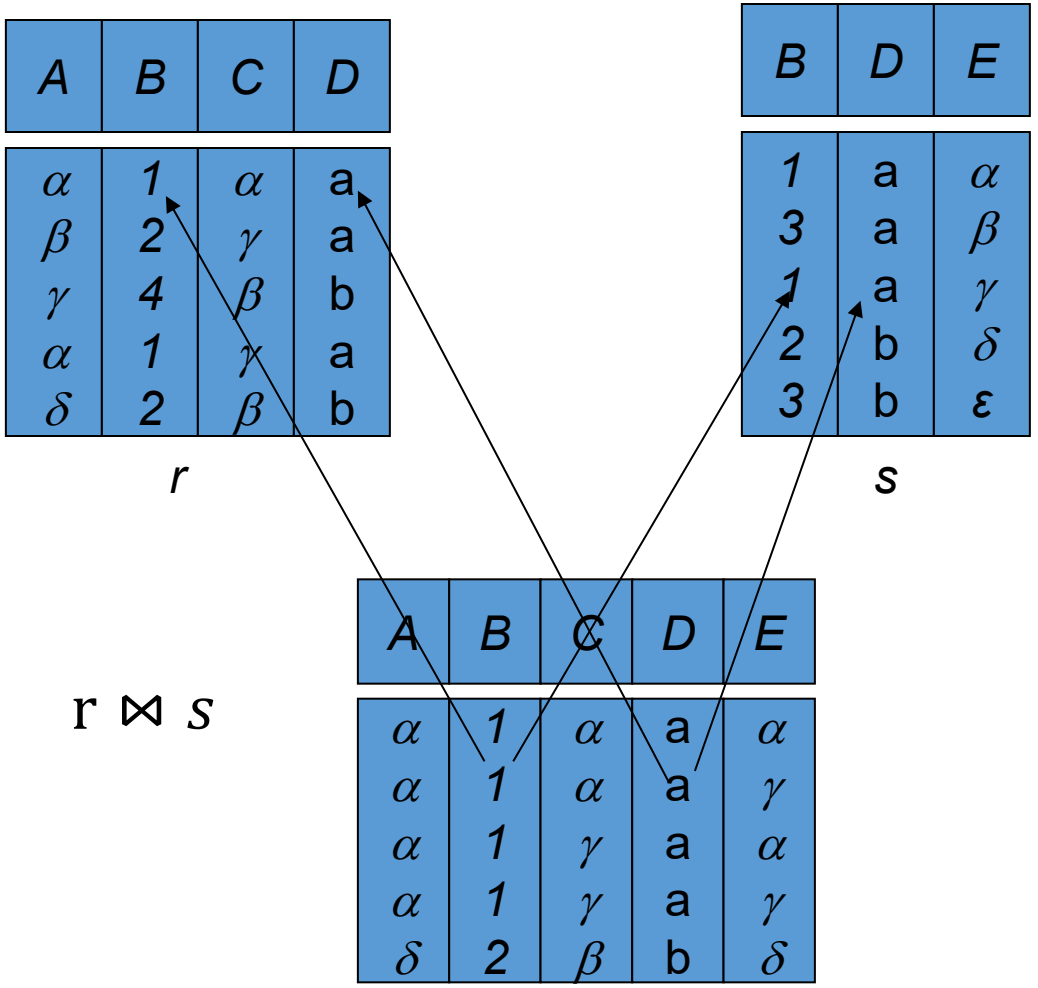
- Relations r, s:



2.2 关系代数

➢专门的关系运算

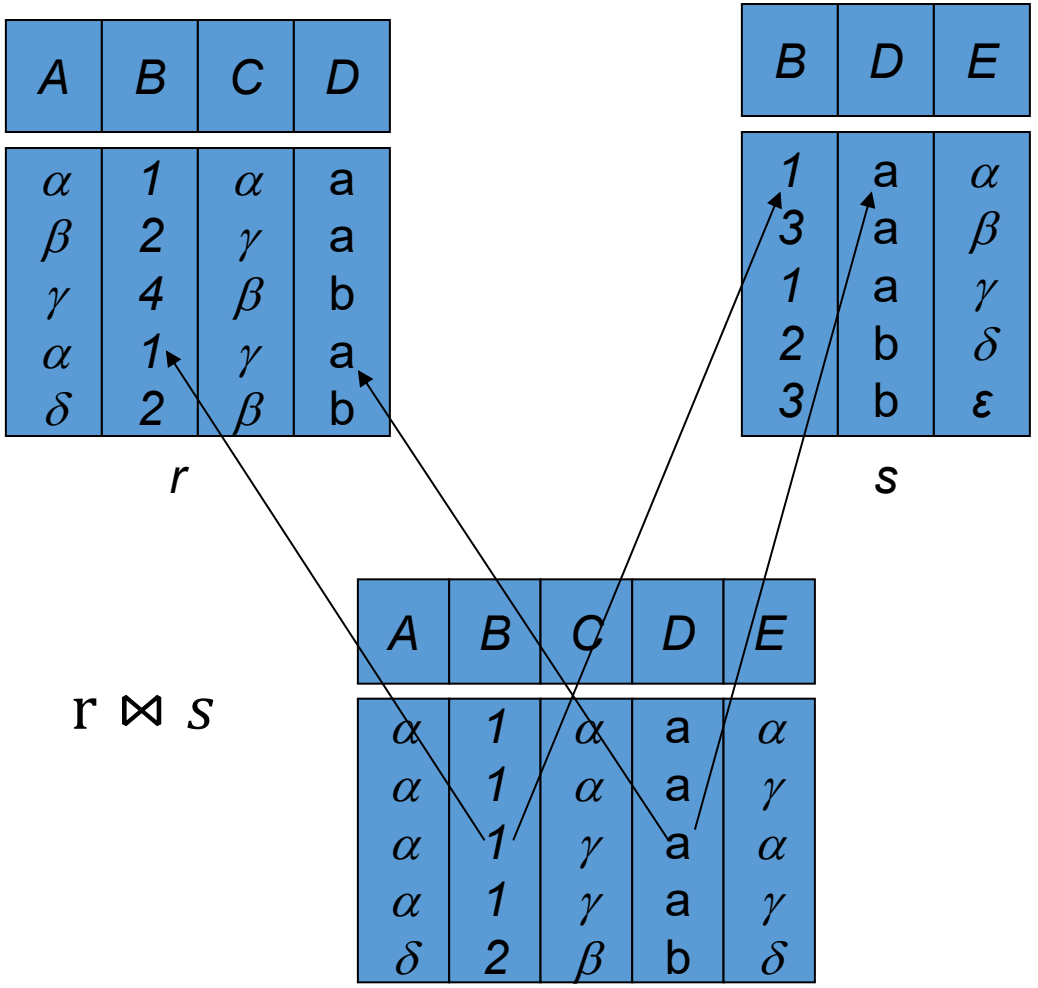
- Relations r, s:



2.2 关系代数

➢专门的关系运算

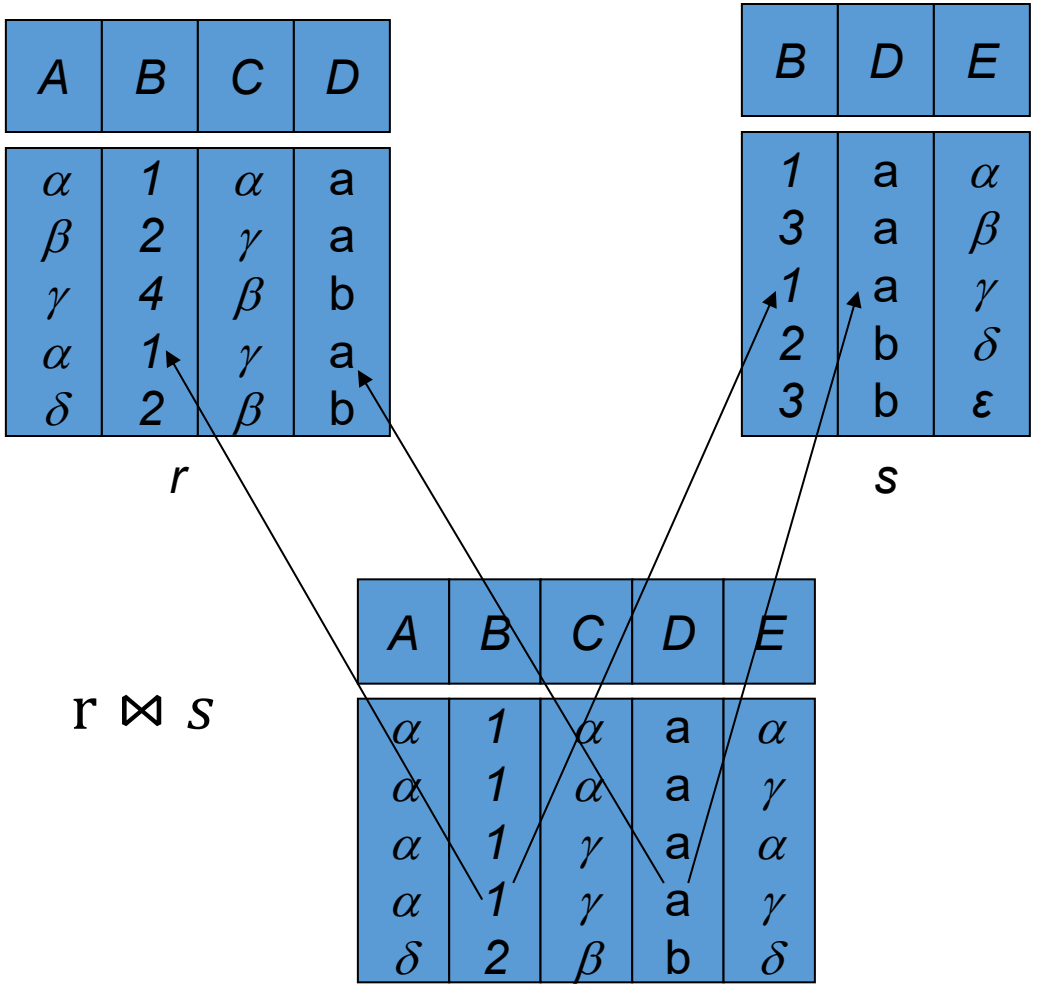
- Relations r, s:



2.2 关系代数

➢专门的关系运算

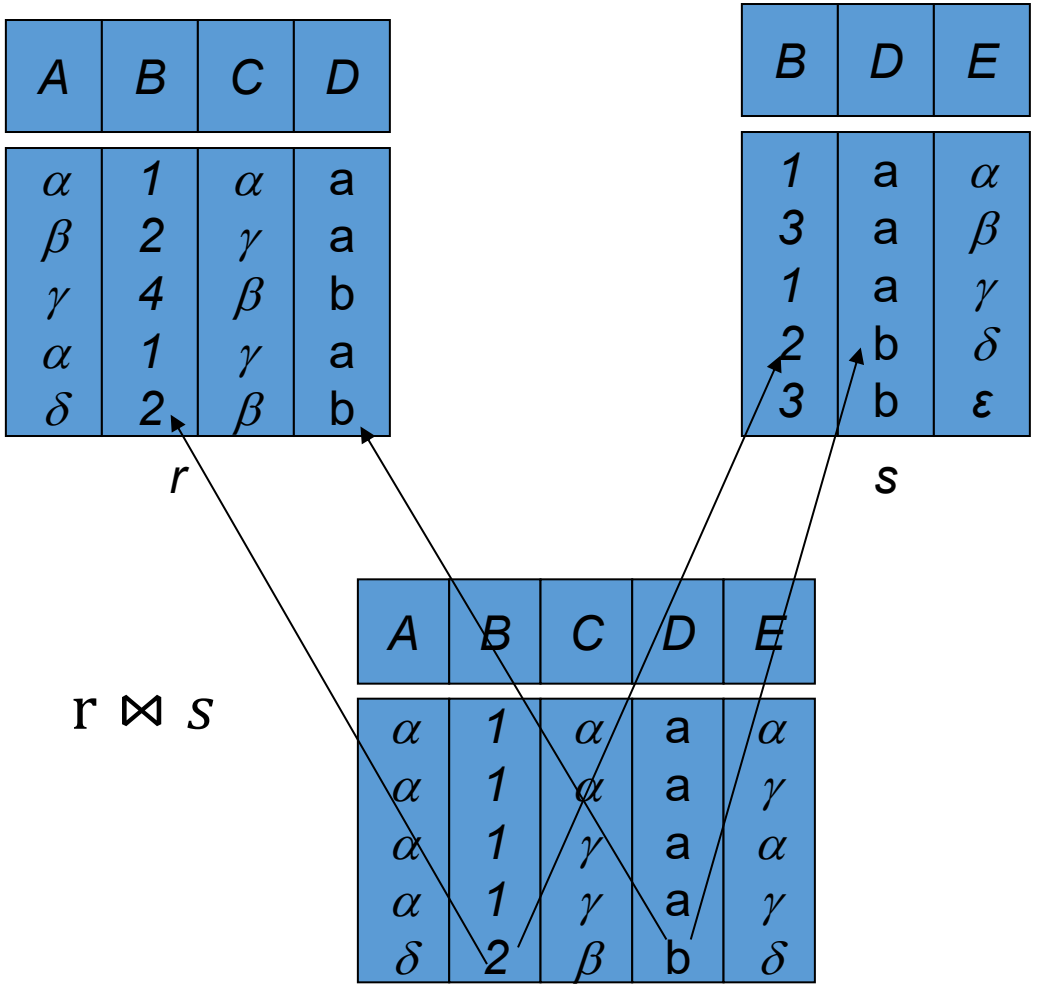
- Relations r, s:



2.2 关系代数

➢专门的关系运算

- Relations r, s:



2.2 关系代数

➢专门的关系运算

R

A	B	C
a1	b1	5
a1	b2	6
a2	b3	8
a2	b4	12

S

B	E
b1	3
b2	7
b3	10
b3	2
b5	2

$R \bowtie S$

A	B	C	E
a1	b1	5	3
a1	b2	6	7
a2	b3	8	10
a2	b3	8	2

$R \bowtie_{C<E} S$

A	R.B	C	S.B	E
a1	b1	5	b2	7
a1	b1	5	b3	10
a1	b2	6	b2	7
a1	b2	6	b3	10
a2	b3	8	b3	10

$R \bowtie_{R.B=S.B} S$

A	R.B	C	S.B	E
a1	b1	5	b1	3
a1	b2	6	b2	7
a2	b3	8	b3	10
a2	b3	8	b3	2

2.2 关系代数

➢专门的关系运算

- Relation schema
  - *Student* (***sno***, name, sex, age)
  - *Course* (***cno***, cname, credit)
  - *SC* (***sno***, ***cno***, grade)

2.2 关系代数

➢专门的关系运算

Student

sno	sname	sex	age
801	Zhang	F	19
802	Li	M	20
803	Wang	M	20
804	Zhao	F	20
805	Qian	m	19

SC

sno	cno	grade
801	01	92
801	02	78
801	03	85
802	01	82
802	02	90
803	01	88

Course

cno	cname	credit
01	DB	4
02	DS	4
03	OS	4

sno	sname	sex	age	cno	cname	credit	grade
801	Zhang	F	19	01	DB	4	92
801	Zhang	F	19	02	DS	4	78
801	Zhang	F	19	03	OS	4	85
802	Li	M	20	01	DB	4	82
802	Li	M	20	02	DS	4	90
803	Wang	M	20	01	DB	4	88

2.2 关系代数

➢专门的关系运算

- 连接（join）
  - 外连接
    - 外连接将不满足连接条件的元组仍然保留在结果关系中，并在缺失属性上取null。根据保留元组的不同，外连接又分为左外连接，右外连接，全外连接

2.2 关系代数

➢专门的关系运算

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

R

B	C	D
b1	c1	d1
b3	c3	d3
b4	c4	d4

S

A	B	C	D
a1	b1	c1	d1
a2	b2	c2	null
a3	b3	c3	d3

$R^*\bowtie S$

A	B	C	D
a1	b1	c1	d1
a3	b3	c3	d3
null	b4	c4	d4

$R \bowtie^* S$

A	B	C	D
a1	b1	c1	d1
a2	b2	c2	null
a3	b3	c3	d3
null	b4	c4	d4

$R^*\bowtie^* S$

2.2 关系代数

➢专门的关系运算

- Course（cno, cname, credit, pcno)
- 列出所有课程名及先修课程名

2.2 关系代数

➢专门的关系运算

- 连接（join）
- 半连接
  - R和S的半连接是将其自然连接的结果在R的属性集上投影，可以表示为：

$$R \ltimes S = \pi_R(R \bowtie S), S \ltimes R = \pi_S(R \bowtie S)$$

- 半连接不具有交换率

2.2 关系代数

➢专门的关系运算

- 连接（join）

A	B	C
a1	b1	c1
a2	b1	c1
a3	b2	c3
a4	b4	c4

R

B	C	D
b1	c1	d1
b1	c1	d2
b2	c2	d2
b4	c4	d3

S

A	B	C
a1	b1	c1
a2	b1	c1
a4	b4	c4

$R \ltimes S$

B	C	D
b1	c1	d1
b1	c1	d2
b4	c4	d3

$S \ltimes R$

2.2 关系代数

➢专门的关系运算

- 除（Divison）
  - 给定两个关系R(X，Y)，S(Y，Z)，其中X，Y均是属性的集合。设T(X) = R(X，Y)÷S(Y,Z)表示R与S除运算的结果，÷是除运算的算符，结果关系T的属性集由X构成，则T与S的笛卡儿积必定是R的一个子集。R÷S可表示如下：

$$T = R \div S = \{t[X] | t \in R \wedge \forall_s \in S, t[X]_s \in R\} = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$$

  - 式中t[X]s表示由元组t在属性子集X上的分量同关系S的元组s构成的新元组

2.2 关系代数

➢专门的关系运算

- 除（Divison）

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d2
a1	b1	c3	d3
a2	b2	c2	d2
a3	b3	c1	d1
a3	b3	c3	d3

R

C	D
c1	d1
c3	d3

S

A	B
a1	b1
a3	b3

R÷S

2.2 关系代数

➢专门的关系运算

- 除（Divison）
  - 在关系R中，A属性可以取{a1, a2,a3,a4}。
    - a1的象集为{(b1,c2),(b2,c3),(b2,c1)}
    - a2的象集为{(b3,c7),(b2,c3)}
    - a3的象集为{(b4,c6)}
    - a4的象集为{(b6,c6)}
  - S在(B,C)上的投影为： {(b1,c2),(b2,c3),(b2,c1)}
  - 显然，只有a1的象集包含了S在(B,C)属性组上的投影，所以R÷S={a1}

结果属性　象集属性

A	B	C
a1	b1	c2
a2	b3	c7
a3	b4	c6
a1	b2	c3
a4	b6	c6
a2	b2	c3
a1	b2	c1

关系R

A
a1

R÷S

B	C	D
b1	c2	d1
b2	c1	d1
b2	c3	d2

关系S

2.2 关系代数

➢关系运算

- 最小完备集，如 $\{\cup, -, \bowtie, \sigma, \pi\}$
- 最小完备集不是唯一的

2.2 关系代数

➢关系代数查询实例

- Student(S#，Class，Sname，Sex，Age)
- Course（C#，Cname，Teacher)
- SC（S#，C#，Grade)
- *例2.1.王老师开设课程的编号及课程名：*

$$\pi_{C\#,Cname}(\sigma_{Teacher='王'}(Course))$$

- *例2.2. 至少选修了一门王老师所开设课程的学生学号：*

$$\pi_{S\#}((\sigma_{Teacher='王'}(SC \bowtie Course))$$

$$\pi_{S\#}(SC \bowtie \pi_{C\#}(\sigma_{Teacher='王'}(Course))$$

2.2 关系代数

➢关系代数查询实例

- *例2.3.李同学未选课程的编号：*

$$\pi_{C\#}(Course) - \pi_{C\#}(\sigma_{Sname='李'}(Student \bowtie SC))$$

$$\pi_{C\#}(Course) - \pi_{C\#}(\sigma_{Sname='李'}(Student) \bowtie SC)$$

- *例2.4.至少选修了两门课程的学生学号：*

$$\pi_1(\sigma_{1=4 \wedge 2 \neq 5}(SC \times SC))$$

- *例2.5.全部学生均选修了的课程编号及名称：*

$$\pi_{C\#,Cname}(Course \bowtie (\pi_{S\#,C\#}(SC) \div \pi_{S\#}(student))$$

2.2 关系代数

➢关系代数查询实例

- *例：检索选修课程名为Maths的学生学号与姓名*

$$\pi_{S\#,Sname}(\sigma_{Cname='Maths'}(S \bowtie SC \bowtie C))$$

- *例：检索选修课程号为C2或C4的学生学号*

$$\pi_{S\#}(\sigma_{C\#='C2' \vee C\#='C4'}(SC))$$

- *例：检索至少选修课程号为C2和C4的学生学号*

$$\pi_{S\#}(\sigma_{1=4 \wedge 2='C2' \wedge 5='C4'}(SC \times SC))$$

  - *这里(SC×SC)表示关系SC自身相乘的笛卡儿积操作。*

2.3 关系演算

➢关系演算的定义

- 关系演算
  - 以数理逻辑中的谓词演算为基础
- 按谓词变元不同 进行分类
  - 元组关系演算：

    以元组变量作为谓词变元的基本对象
  - 域关系演算：

    以域变量作为谓词变元的基本对象

2.3 关系演算

➢元组关系演算

- 元组关系演算表达式的一般形式是$\{t|\ P(t)\}$，其中t是元组变量，$P(t)$是元组关系演算公式，一般由原子公式构成

2.3 关系演算

➢元组关系演算

- 原子公式有以下的三种形式：
  - $R(t)$，t是元组变量，R是关系，$R(t)$表示$t \in R$，即t是R的一个元组；
  - $s[i]\theta t[j]$，s和t都是元组变量，$s[i]$表示元组s的第i个分量，$t[j]$表示t的第j个分量，θ是算术比较运算符。$s[i]\theta t[j]$表示s的第i个分量和t的第j个分量满足θ比较关系。
  - $s[i]\theta c$或$c\theta s[i]$，c是常量，$s[i]\theta c$表示元组s的第i个分量和常量c满足θ这样的比较关系。

2.3 关系演算

➢元组关系演算

- 在原子公式的基础上可以通过以下递归定义构成元组关系演算公式：
  - 原子公式也是公式；
  - 如果P1，P2是公式，则P1∧P2，P1∨P2， P1也是公式，分别表示：

    如果P1，P2同时为真，则P1∧P2为真，否则P1∧P2为假；

    如果P1，P2中至少有一个为真，则P1∨P2为真，否则P1∨P2为假；

    如果P1为真，则 P1为假，如果P1为假，则 P1为真；
  - 如果P是公式，则∃t(P)也是公式。 ∃是存在量词符号，∃t(P)表示当至少存在一个t使得P为真，则 ∃t(P)为真，否则为假；
  - 如果P是公式，则 ∀t(P)也是公式。∀是全称量词符号，∀t(P)表示如果所有的t都使P为真，则 ∀t(P)为真，否则为假；

2.3 关系演算

➢元组关系演算

- 在元组关系演算公式中，各类算符优先级为：
  - 算术比较运算符优先级最高；
  - 量词∃和∀优先级其次，且∃的优先级高于∀；
  - 逻辑运算符优先级最低，且┐的优先级高于∧，∧的优先级高于∨；
  - 括号可以改变算符的优先级，加括号时，括号内算符优先，同一括号内算符优先级遵循上述各项。
- 有限次使用上述规则得到的即是元组关系演算公式，除此之外的构成形式不是元组关系演算公式

2.3 关系演算

➢元组关系演算

A	B	C
1	2	3
4	5	6
7	8	9

R

D	E
3	4
7	6

S

A	B	C
4	5	6
7	8	9

A	B	E
1	2	4
1	2	6
4	5	6

$R1=\{t|(\exists u)(R(t)\wedge S(u)\wedge t[3]>u[1])\}$

$R2=\{t|(\exists u)(\exists v)(R(u)\wedge S(v)\wedge u[2]<v[1]\wedge t[1]=u[1]\wedge t[2]=u[2]\wedge t[3]=v[2])\}$

2.3 关系演算

➢元组关系演算

- 基本关系代数运算的元组关系演算等价表示：
  - ∪

$$R \cup S = \{t | R(t) \vee S(t)\}$$

  - −

$$R - S = \{t | R(t) \wedge \neg S(t)\}$$

  - ×

$$R \times S = \{t^{(m+n)} | (\exists u^{(m)})(\exists u^{(n)})(R(u) \wedge S(v) \wedge t[1] = u[1] \wedge \cdots t[m] = u[m] \wedge t[m+1] = v[1] \wedge \cdots \wedge t[m+n] = u[n])\}$$

其中$t^{(m+n)}$表示t是$m+n$元关系的元组。

2.3 关系演算

➢元组关系演算

- 基本关系代数运算的元组关系演算等价表示：
  - σ
    - $\sigma_F(R) = \{t|R(t) \wedge F'\}$，其中$F'$是选择条件$F$在元组关系演算中的等价表示。
  - π

$$\pi_{i1,i2,\cdots,ik}(R) = \{t^{(k)}|(\exists u)(R(u) \wedge t[1] = u[i1] \wedge \cdots \wedge t[k] = u[ik])\}$$

2.3 关系演算

➢元组关系演算

- 不产生无限关系和无穷验证的表达式称为安全表达式，为了保证运算的安全性而采取的限制措施称为安全约束
  - 不安全表达式

$$\{t|\neg(t \in Employee)\}$$

    - 问题：若未限定域，结果可能是所有不属于 Employee 的元组，导致无限结果。
  - 安全表达式

$$\{t|t \in Employee \wedge t.salary > 5000\}$$

    - 原因：t 被显式限制在有限的 Employee 关系中。

$$\{t|\exists s \in Student(s.grade =' A' \wedge t.name = s.name)\}$$

    - 原因：存在量词 ∃s 被限定在 Student 关系中，结果仅涉及有限元组。

2.3 关系演算

➢元组关系演算

- 不产生无限关系和无穷验证的表达式称为安全表达式，为了保证运算的安全性而采取的限制措施称为安全约束
  - 安全约束有限集合DOM
    - $DOM(\Psi)$是一个有限集合，其中的每个符号要么是Ψ中明显出现的符号，要么是出现在Ψ中的某个关系R的某元组的分量。
    - DOM主要用于约束Ψ中一些谓词的计算范围，它不必是最小集合。
  - *例如，下图关系R，令*$S = \{t|\neg R(t)\}$*，则S可能是无限关系，定义一个安全域*

$$DOM = \pi_A(R) \cup \pi_B(R) \cup \pi_C(R) = \{\{a1, a2\}, \{b1, b2\}, \{c1, c2\}\}$$

R

A	B	C
a1	b1	c1
a2	b2	c2

2.3 关系演算

➢元组关系演算

- 安全表达式的构造
  - 规则1：所有自由变量必须被显式约束
    - 例如，$\{t \mid t \in R \wedge P(t)\}$ 是安全的，因为 t 被限制在有限的关系 R 中。
  - 规则2：量词必须限定在有限范围内
    - 存在量词（∃）：应写成 $\exists t \in S\,(P(t))$，而非 $\exists t\,(P(t))$。
    - 全称量词（∀）：同理，需限定范围，如 $\forall t \in S\,(P(t))$。
  - 规则3：否定必须作用在有限集合上
    - 例如，$\{t \mid t \in R \wedge \neg(t.age < 18)\}$ 是安全的，因为 t 被约束在有限的关系R 中，否定操作仅在有限范围内生效。

2.3 关系演算

➢元组关系演算

- 安全表达式的构造
  - *例2.6. 查询“离散数学”课程的编号及任课教师*

$$R = \{t \mid (\exists u)(Course(u) \wedge u[2] = \text{'离散数学'} \wedge t[1] = \quad u[1] \wedge t[2] = u[3])\}$$

  - *例2.7. 选修了‘数据结构’的学生学号及姓名*

$$R = \{t \mid (\exists u)(\exists v)(\exists w)\ (Student(u) \wedge SC(v) \wedge Course(w) \wedge u[1] = v[1] \wedge v[2] = w[1] \wedge w[2] = \text{'数据结构'} \wedge t[1] = u[1] \wedge t[2] = u[3])\}$$

2.3 关系演算

➢元组关系演算

- 安全表达式的构造
  - 例：检索选修课程号为C2或C4的学生学号

$$\pi_{S\#}(\sigma_{C\#='C2'\vee C\#='C4'}(SC))$$

$$\{t|(\exists u)(SC(u)\wedge(u[2]=\vee u[2]='C4')\wedge t[1]=u[1]\}$$

  - 例：检索至少选修课程号为C2和C4的学生学号

$$\pi_{S\#}(\sigma_{1=4\wedge 2='C2'\wedge 5='C4'}(SC\times SC))$$

$$\{t|(\exists u)(\exists v)(SC(u)\wedge SC(v)\wedge u[2]='C2'\wedge v[2]='C4'\wedge u[1]=v[1]\wedge t[1]=u[1])\}$$

2.3 关系演算

➢域关系演算

- 域关系演算表达式的一般形式是$\{x1 \ldots xk \mid P(x1, \ldots, xk)\}$，其中$x1, \ldots, xk$都是域变量，$P(x1, \ldots, xk)$是域关系演算公式，一般由原子公式构成

2.3 关系演算

➢域关系演算

- 类似元组演算，域演算原子公式定义如下：
  - $R(x1，\ \ldots，\ xk)$，其中$x1，\ \ldots，\ xk$都是域变量或常量，R是k元关系，$R(x1，\ \ldots，\ xk)$表示由$x1，\ \ldots，\ xk$构成的元组满足关系R；
  - $x\theta y$，其中x和y都是域变量，θ是算术比较运算符。 $x\theta y$表示x和y满足θ比较关系；
  - $x\theta c$或$c\theta x$，其中x是域变量，c是常量，$x\theta c$表示变量x与常量c满足θ比较关系。$x\theta c$和$c\theta x$是等价的。

2.3 关系演算

➢域关系演算

- 在原子公式的基础上域演算公式递归定义如下：
  - 原子公式也是公式；
  - 如果P1，P2是域演算公式，则$P1 \wedge P2$，$P1 \vee P2$，　$P1$也是域演算公式；
  - 如果P是域演算公式，则 $\exists x(P)$也是域演算公式；
  - 如果P是域演算公式，则 $\forall x(P)$也是域演算公式；
  - 域演算公式中各类算符优先级与元组演算相同；
  - 有限次使用上述规则得到的即是域演算公式，除此之外的构成形式不是域演算公式。

2.3 关系演算

➢域关系演算

A	B	C
1	2	3
4	5	6
7	8	9

R

D	E
3	4
7	6

S

A	B	C
1	2	3
4	5	6
7	8	9

$R1=\{xyz|\ (R(xyz) \wedge y<10)\}$

B	C	D
5	6	3
8	9	3
8	9	7

$R2=\{xyz|\ (\exists u)(\exists v)(\ R(uxy) \wedge S(zv) \wedge u \geq v)\}$

2.3 关系演算

➢域关系演算

- 例：检索学习课程号为C2的学生学号与成绩

$$\{t_1 t_2 | (\exists_{u_1 u_2 u_3})(SC(u_1 u_2 u_3) \wedge u_2 =' C2' \wedge t_1 = u_1 \wedge t_2 = u_3\}$$

可化简为$\{t_1 t_2 | SC(t_1' C2' t_2)\}$

- 例：检索学习课程号为C2的学生学号与姓名

$\{t_1 t_2 \mid (\exists u_1 u_2 u_3 u_4)\ (\exists v_1 v_2 v_3)(S(u_1 u_2 u_3 u_4)\ \wedge SC\ (v_1 v_2 v_3) \wedge v_2 =\ 'C2'\ \wedge u_1 = v_1 \wedge\ t_1 = u_1 \wedge t_2 = u_2)\}$

可简化为$\{t_1 t_2 \mid (\exists u_3 u_4)\ (\exists v_3)(S(t_1 t_2 u_3 u_4)\ \wedge\ SC(t_1 'C2' v_3)\ \}$